

ors:

as.

```
graph LR; A["{ }"] --- B["[ ]"]; A --- C["{ }"]; A --- D["]]; B --- E["{ = }"]; B --- F["] )"]; B --- G["[ = ]"]; B --- H["[ ]"]; C --- I["{ = }"]; C --- J["[ ]"]; C --- K["{ = }"]; D --- L["[ = ]"]; D --- M["[ = ]"]; D --- N["{ = }"];
```

Gemini Notebook

The Target Sandbox: db.students

Name	Age	City	Salary	Hobbies
nitan	32	kathmandu	50000	['coding', 'reading']
ram	25	pokhara	35000	['football', 'travel']
bob	15	butwal	10000	null

All tactical queries in this playbook run against this specific collection structure. Memorize the fields: we will be filtering by exact matches, age thresholds, and array contents.

The Limit of Exact Matches

General Search

```
db.students.find({  
  city: 'kathmandu'  
})
```

Outcome: Returns Nitán, Gita, Sophia.

Status: Easy, but limited.

The Problem

Find all students older than 18.

```
db.students.find({  
  age: > 18  
})
```

✗ Syntax Error

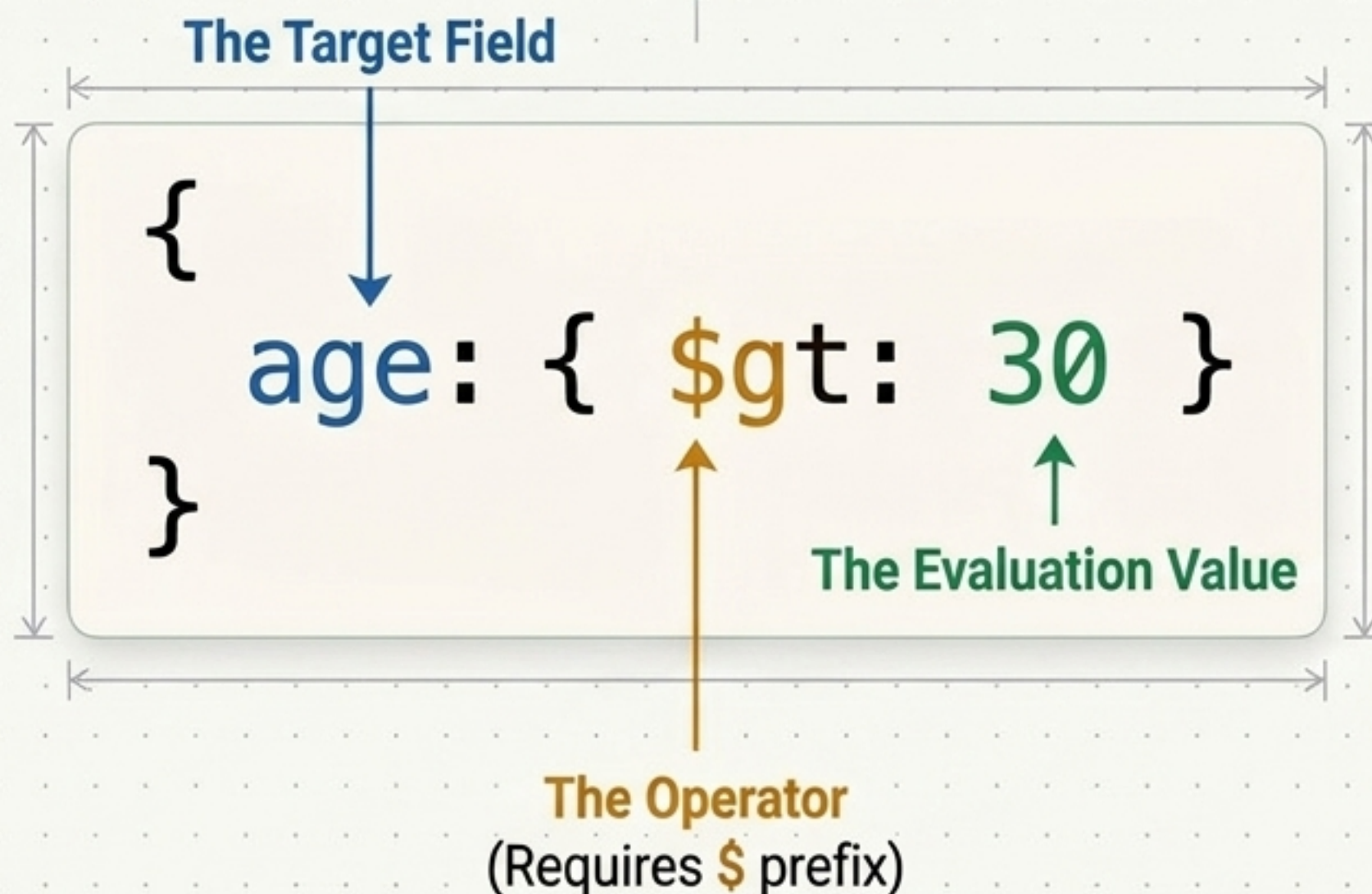
Outcome: Standard math operators do not work in standard JSON queries.

To filter by ranges, inequalities, or array matches, we must translate standard logic into MongoDB Comparison Operators.

The Anatomy of an Operator (The Golden Rule)

The Golden Rule Box:

Prefix the operator with a dollar sign (\$).



Operators cost **MONEY** (\$), and they live inside their own **HOUSE** ({ }).

Always wrap your operator and value in a nested object.

The Operator Matrix: Equivalence & Magnitude

Equivalence (Matching exactness)	Magnitude (Numeric thresholds)	Inclusive Magnitude (Thresholds + exact)
= \$eq : Equal to find({name: {\$eq: 'nitan'}})	> \$gt : Greater Than find({age: {\$gt: 30}})	≥ \$gte : Greater Than Equals find({age: {\$gte: 30}})
≠ \$ne : Not Equal to find({name: {\$ne: 'nitan'}})	< \$lt : Less Than find({age: {\$lt: 30}})	≤ \$lte : Less Than Equals find({age: {\$lte: 30}})

Mnemonic: Read the magnitude operators like simple acronyms. **Greater Than Equals = G.T.E.** Always prefix with **\$**.

The Operator Matrix: Arrays & Math

The Matcher (\$in)



Matches any value contained in an array.

```
find({name: {$in: ['nitan', 'ram']}})
```

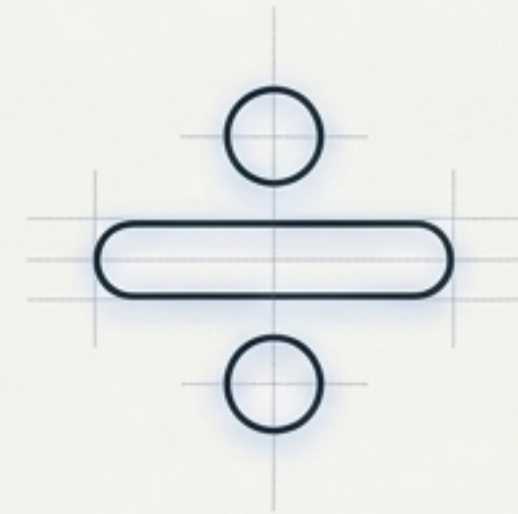
The Excluder (\$nin)



Matches none of the values specified in an array.

```
find({name: {$nin: ['nitan', 'ram']}})
```

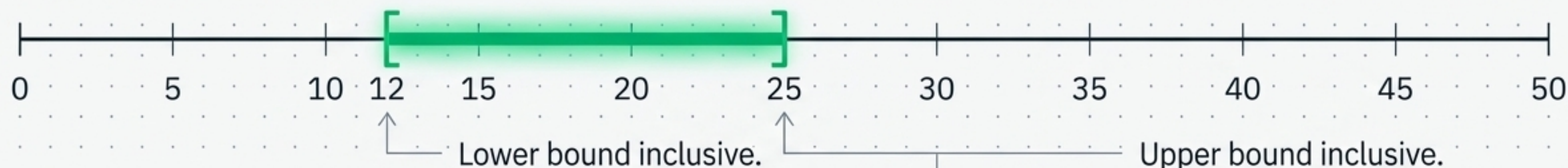
The Calculator (\$mod)



Performs a modulo operation to find remainders.

```
find({age: {$mod: [5, 0]}})
```

Tactical Execution: Range Queries



```
db.students.find({ age: { $gte: 12, $lte: 25 } })
```

Combining Forces: You can pass multiple comparison operators into the same field's "house" (`{ }`) to establish strict boundaries. This is the standard pattern for price ranges, date windows, and age limits.

Tactical Execution: The \$mod Decoder

{ age: { \$mod: [5, 0] } }

Divisor

(What we divide by)

Remainder

(The exact leftover we want)

Translation: Find all students whose age, when divided by 5, leaves a remainder of exactly 0.

Real-World Use Case

Used heavily for pagination, A/B testing user IDs, or cyclical scheduling logic.

Examiner Trap #1: The \$in Pitfall

```
{  
  name: { $in: 'nitan' } }  
}
```



Result: CRASH / UNEXPECTED BEHAVIOR.

Reason: String passed where an array is required.

```
{  
  name: { $in: ['nitan', 'ram'] }  
}
```

Result: SUCCESS.

Reason: Values properly enclosed in brackets.



\$IN needs an **IN**-ventory. Always use an Array **[]**.

Viva Simulator: Round 1 (Rapid Fire)



Examiner: How would you find users over a certain age limit (e.g., 18)?

Riesp: { age: { \$gte: 18 } }

Examiner: How would you find students whose age is exactly 25 or 30?

Response: { age: { \$in: [25, 30] } }

Instructor Note: In a Viva, don't just say the operator. Recite the nested object structure to prove you know the exact syntax.

Viva Simulator: Round 2 (The Curveball)

Dissection

Examiner: Can you use multiple comparison operators on the same field?

Yes. We can declare a single field and pass a nested object containing multiple operators.

```
db.students.find({ age: { $gte: 18, $lte: 30 } })
```

Nailing this question demonstrates that you understand MongoDB views the field as the key, and the entire operator logic as a unified value object.

The 30-Second Interview Cheat Sheet

Operator	Plain English Translation	Example Syntax
\$eq	Matches exactly	<code>{age: {\$eq: 25}}</code>
\$ne	Not equal to	<code>{city: {\$ne: 'kathmandu'}}</code>
\$gt	Greater than	<code>{salary: {\$gt: 30000}}</code>
\$gte	Greater than/equal	<code>{age: {\$gte: 18}}</code>
\$lt	Less than	<code>{age: {\$lt: 65}}</code>
\$lte	Less than/equal	<code>{salary: {\$lte: 50000}}</code>
\$in	Matches any in array	<code>{city: {\$in: ['pokhara', 'butwal']}}</code>
\$nin	Matches none in array	<code>{name: {\$nin: ['bob', 'john']}}</code>
\$mod	Division remainder	<code>{age: {\$mod: [5, 0]}}</code>

Screenshot this blueprint. Remember the \$, remember the nested { }.

Mission Complete & Resources

Further Self-Study

- [Official MongoDB Manual](#)  Deep dive into aggregation and indexing.
- [MongoDB Node.js Driver Documentation](#)  Implementing these operators in backend code.

Source Authority

The JavaScript examples and tactical query structures in this playbook are credited to the original author of the repository MongoDB Learning Site.

Compile your knowledge. Run the code. Ace the Viva.